Module III

Consensus algorithms, Crash fault-tolerance (CFT) algorithms - Paxos, raft. Byzantine fault-tolerance (BFT) algorithms - Practical Byzantine Fault tolerance (PBFT), Proof of Work (PoW), Proof of stake (PoS), Types of PoS.

Bitcoin - Definition, Cryptographic keys - Private keys, Public keys, addresses. Transactions- Lifecycle, coinbase transactions, transaction validation. Blockchain - The genesis block. Mining - Tasks of miners, mining algorithm, hash rate. Wallets - Types of wallets.

Crash Fault

Nodes or replicas may experience downtime at any time, stop running for a short time and recover later.

The network may be interrupted at any time.

A sent message may be lost during delivery and cannot be received.

A sent message may be delayed and received after a long time.

Messages may experience the out-of-order problem during the delivery process.

The network may be divided into sub networks.

The Byzantine Fault

Malicious nodes in networks may change and forge data at any time.

These troublemaking problems that may change and forge data or response information are called Byzantine faults.

Nodes misbehave - Send different information to different peers.

Byzantine Generals Problem

By analogy, some number of generals plan an attack on a castle together and each general can choose to start the attack or retreat.

However, to successfully take the castle, all the generals must act synchronously.

Next, given that the generals are too far from each other to use direct communication, messengers are used to carry messages.

However, messages are not reliable. They may successfully delivery messages after a very long time, fail to deliver messages or even change with messages.

The generals may not be reliable, either, for example, one of them may be a traitor who does not act in accordance with the plan.

The messengers in this story represent communication channels in distributed networks and the generals represent nodes.

Four Byzantine Generals Problem: Commander Faulty

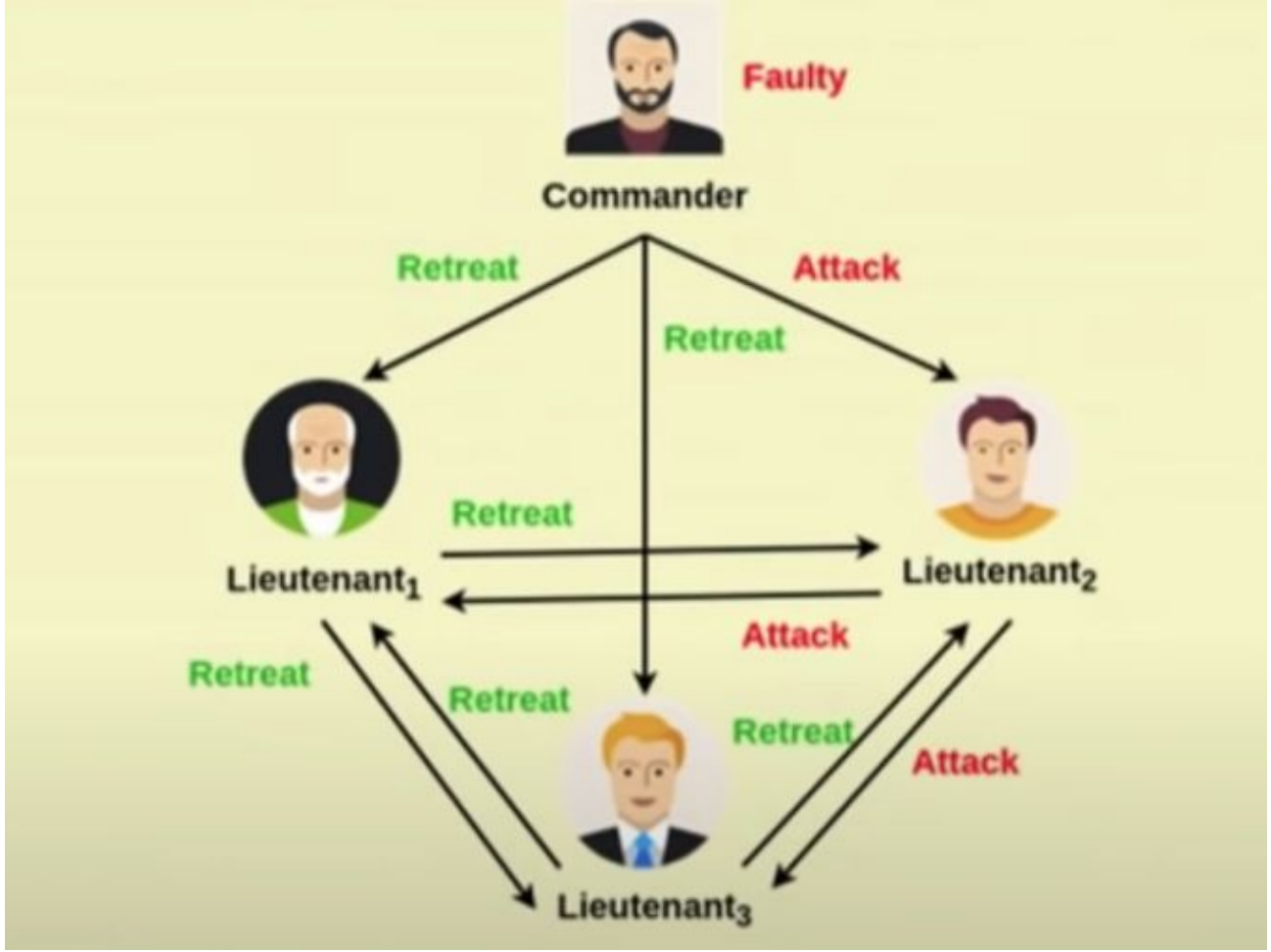
In this case, there are four Byzantine Generals, out of which three are non-faulty Lieutenants and a faulty Commander.

They communicate with each other using the message passing technique.

The Commander sends the differing messages to Lieutenants, and the Lieutenants exchange the received message.

When all the correct Lieutenants follow the majority voting principle, although Commander is a Byzantine node, Lieutenant (1), Lieutenant (2) & Lieutenant (3) can correctly decode the message.

In the Byzantine Generals Model, If there are F faulty nodes (crash fault), we need at least 2F+1 nodes to reach consensus.



<u>Consensus based on majority voting</u>

Lieutenant (1) = {Retreat, Attack, Retreat) = Retreat

Lieutenant (2) = {Retreat, Attack, Retreat} = Retreat

Lieutenant (3) = {Attack, Retreat, Retreat) = Retreat

Fault Tolerance

Algorithms used to solve crash faults are called Crash Fault Tolerance (CFT) algorithms or non-Byzantine fault tolerance algorithms.

If a system is in a reliable internal network, we only need to consider the issue of crash fault tolerance (CFT).

Byzantine faults may cause unauthorized changes, have higher complexity and are more difficult to solve.

Algorithms for solving these problems are called Byzantine Fault Tolerance (BFT) algorithms.

CFT Consensus

Paxos (Proposed by Lamport, the most fundamental CFT)

RAFT (Much simpler than Paxos)

Paxos

Paxos is an algorithm that is used to achieve consensus among a distributed set of computers that communicate via an asynchronous network.

One or more clients proposes a value to Paxos and we have consensus when a majority of systems running Paxos agrees on one of the proposed values.

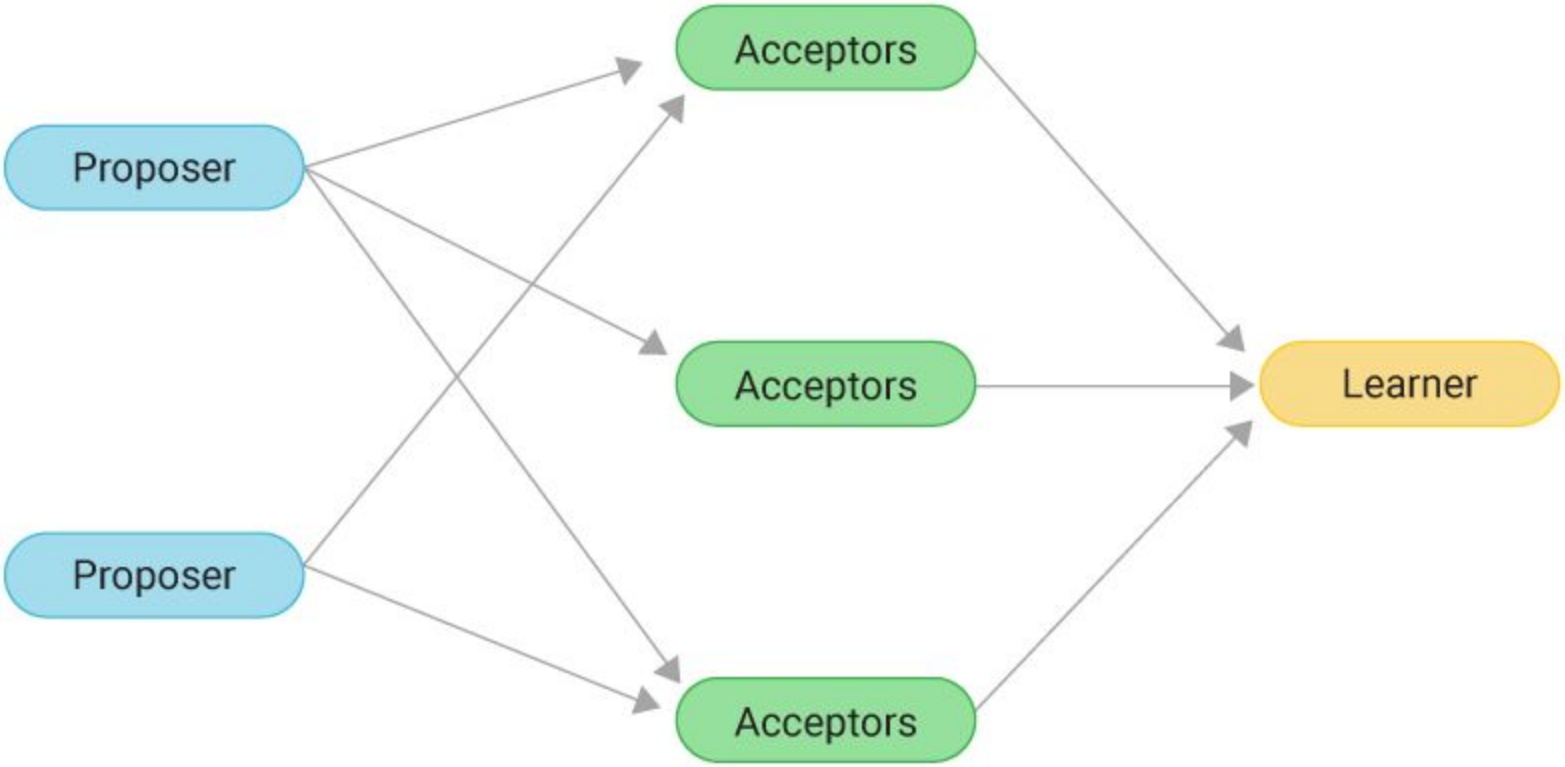
Nodes can take the roles of,

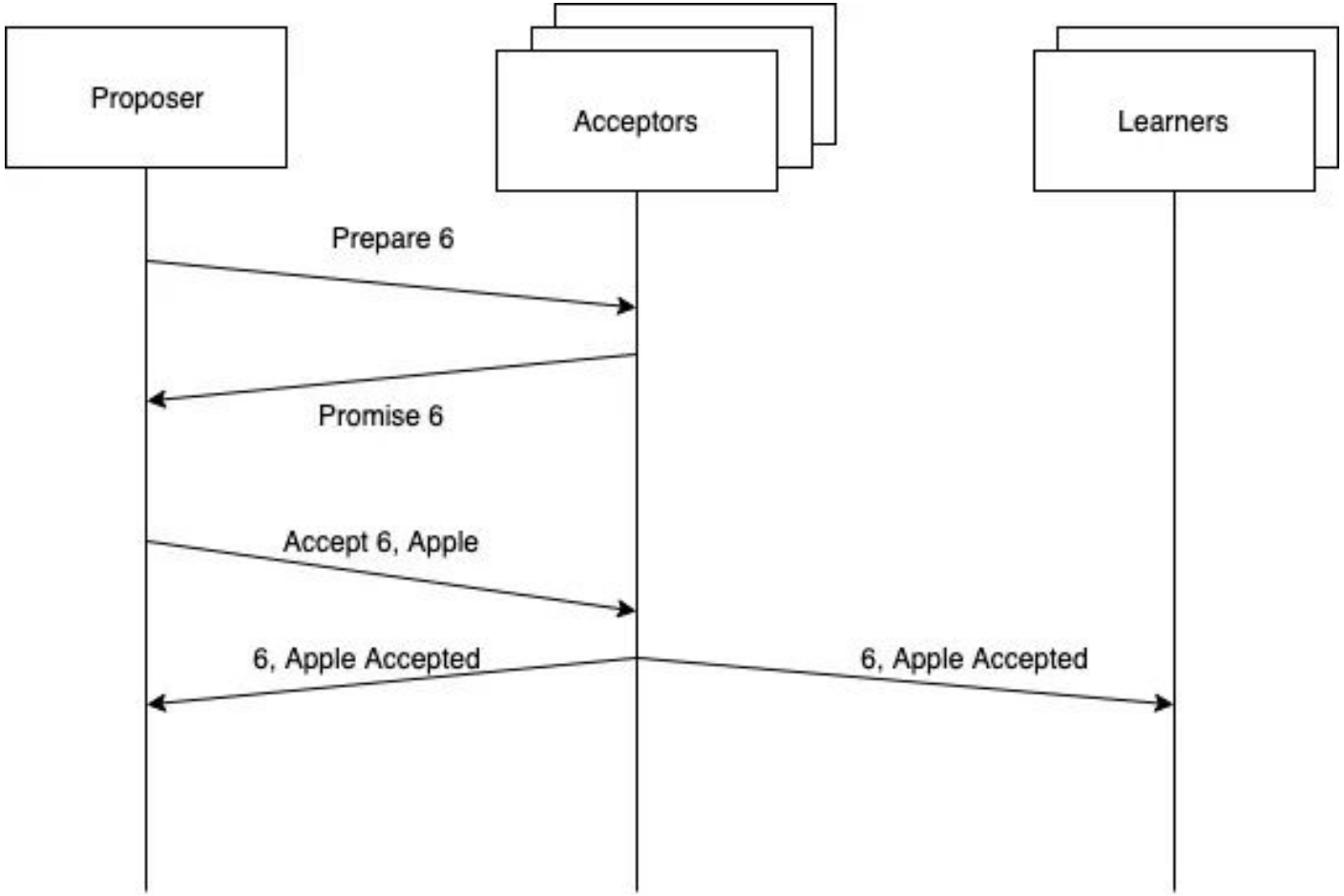
Proposers, who receive requests (values) from clients and try to convince acceptors to accept the value they propose. Leader is a distinguished proposer

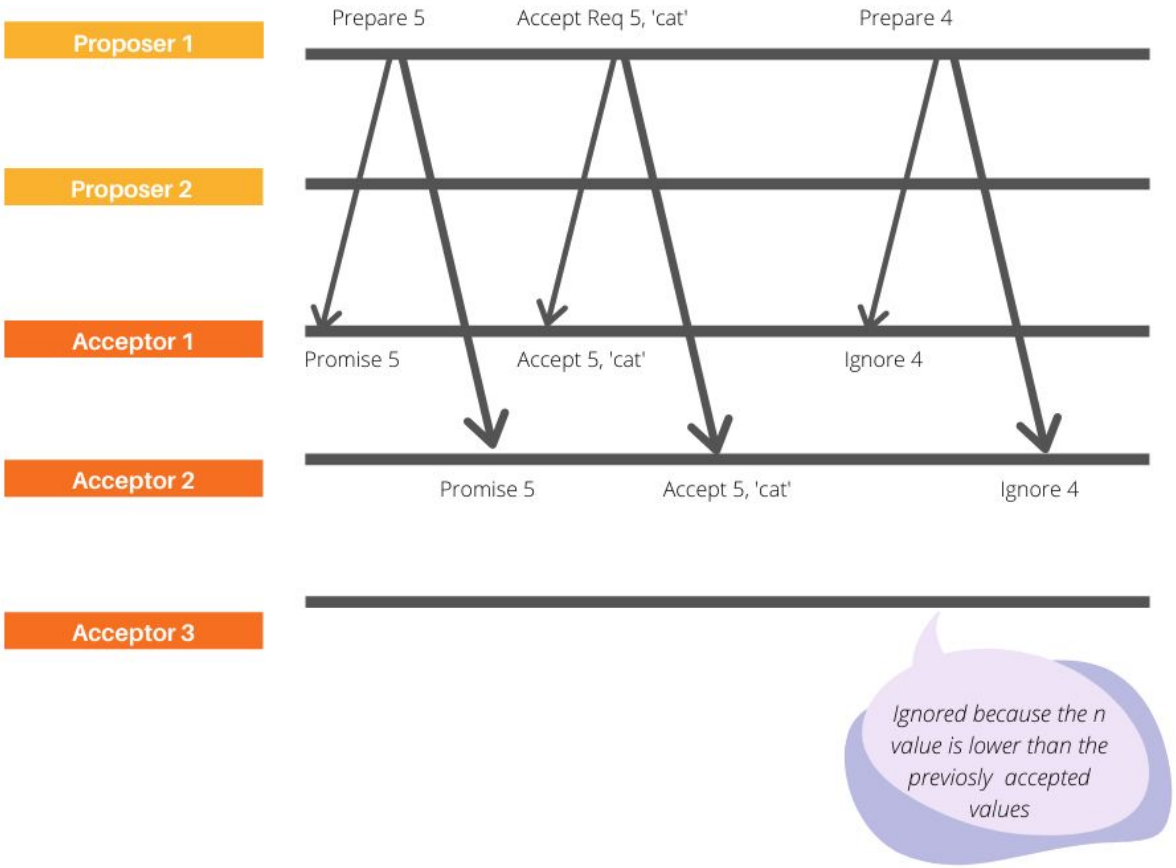
Acceptors, who accept certain proposed values from proposers and let proposers know whether a different value was accepted.

A response from an acceptor represents a vote for a particular proposal. There should be a Quorum of acceptors to complete a round.

Learners, who announce the outcome to all participating nodes.


Acceptors
Proposer
Acceptors
Learner
Proposer
Acceptors


Proposer
Acceptors
Learners
Prepare 6
Promise 6
Accept 6, Apple
6, Apple Accepted
6, Apple Accepted


Proposer 1
Proposer 2
Acceptor 1
Acceptor 2
Acceptor 3
Prepare 5
Accept Req 5, 'cat'
Prepare 4
Promise 5
Accept 5, 'cat'
Ignore 4
Promise 5
Accept 5, 'cat'
Ignore 4
Ignored because the n value is lower than the previosly accepted values

Phase 1: PREPARE-PROMISE

A proposer receives a consensus request for a VALUE from a client.

It creates a unique proposal number, ID, and sends a PREPARE(ID) message to at least a majority of acceptors.

Each acceptor that receives the PREPARE message looks at the ID in the message and decides:

is this ID bigger than any round I have previously received?

If yes

store the ID number, max_id = ID

respond with a PROMISE message

If no

do not respond (or respond with a "fail" message)

If the proposer receives a PROMISE response from a majority of acceptors, it now knows that a majority of them are willing to participate in this proposal.

The proposer can now proceed with getting consensus.

Each of these acceptors made a promise that no other proposal with a smaller number can make it to consensus.

Phase 2: ACCEPT-REQUEST – ACCEPT

If a proposer received a PROMISE message from the majority of acceptors, it now has to tell the acceptors to accept that proposal. If not, it has to start over with another round of Paxos.

In this phase of the protocol, the proposer tells all the acceptors (that are live) what value to accept. It sends:

ACCEPT-REQUEST(ID, VALUE)

to a majority or all of the acceptors. Each acceptor now decides whether to accept the proposal. It accepts the accept request if the ID number of the accept-request is still the largest one that it has seen. Recall that an acceptor promised not to accept proposals from PREPARE messages with smaller numbers but can and will accept proposals with higher numbers. The logic the acceptor uses is:

is the ID the largest I have seen so far, max_id == N?

if yes

    reply with an ACCEPT message & send ACCEPT(ID, VALUE) to all learners

if no

    do not respond (or respond with a "fail" message)

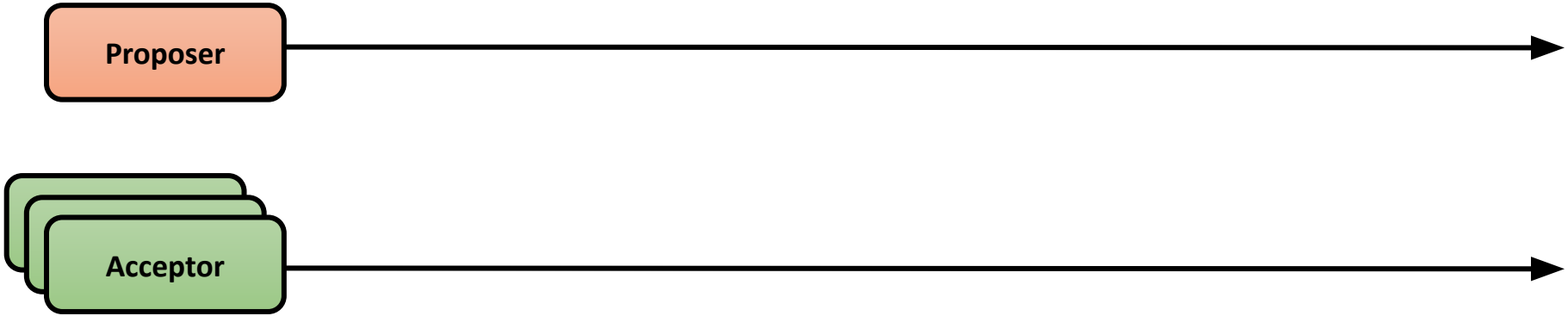
The ACCEPT message can be sent back to the proposer as well as to the learners so they can propagate the value to wherever its action is needed (e.g., append to a log, modify a replicated database, …).

When the proposer or learner receives a majority of accept messages then it knows that consensus has been reached on the value.

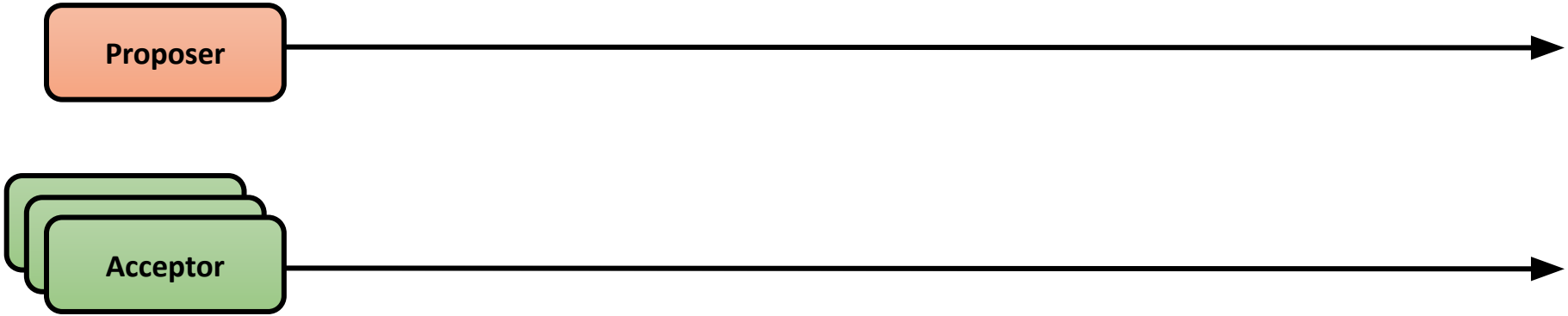
Paxos does not work with Byzantine faults.

So it is a CFT (Crash Fault Tolerant) algorithm.

Paxos Algorithm



Paxos Algorithm



Proposer wants to propose its choice (values):

- Sends PREPARE IDp to a majority (or all) of the **acceptors**

Paxos Algorithm

Proposer wants to propose its choice (values):

- Sends PREPARE IDp to a majority (or all) of the **acceptors**

Paxos Algorithm

Proposer wants to propose its choice (values):

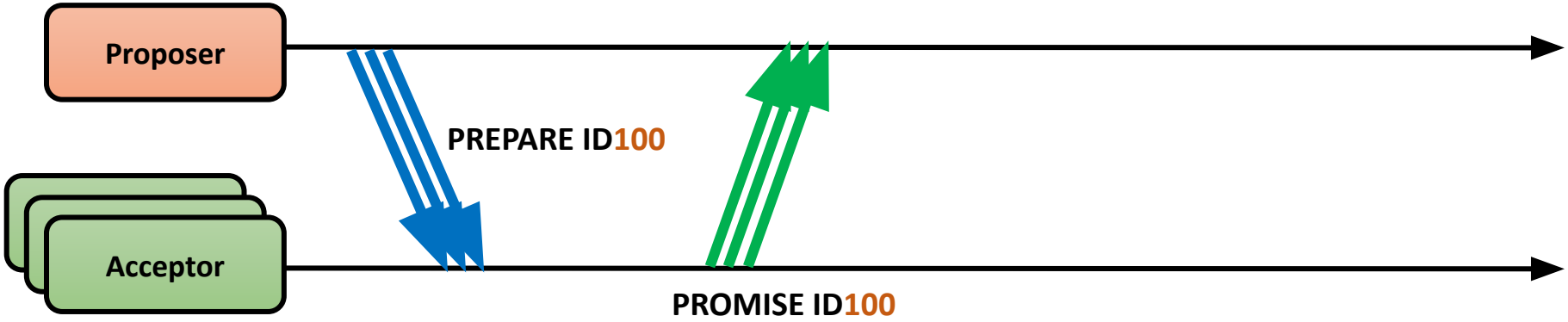
- Sends PREPARE IDp to a majority (or all) of the **acceptors**

Paxos Algorithm

Acceptor received a PREPARE message with IDp:

- Did it promised to ignore requests with this IDp?
  - **YES:** Ignore
  - **NO:** Will promise to ignore any request lower than IDp
    - (?) Reply with PROMISE IDp

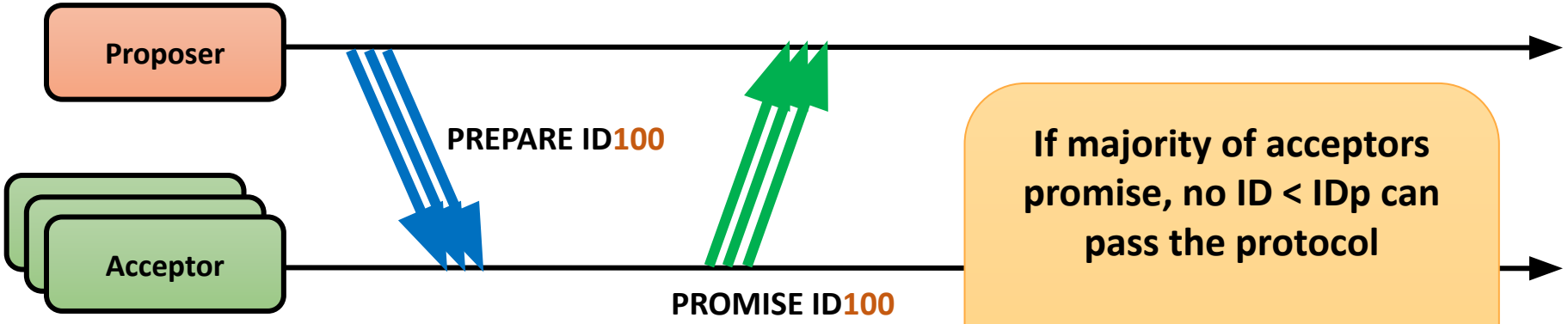
Paxos Algorithm



Acceptor received a PREPARE message with IDp:

- Did it promised to ignore requests with this IDp?
  - **YES:** Ignore
  - **NO:** Will promise to ignore any request lower than IDp
    - (?) Reply with PROMISE IDp

Paxos Algorithm



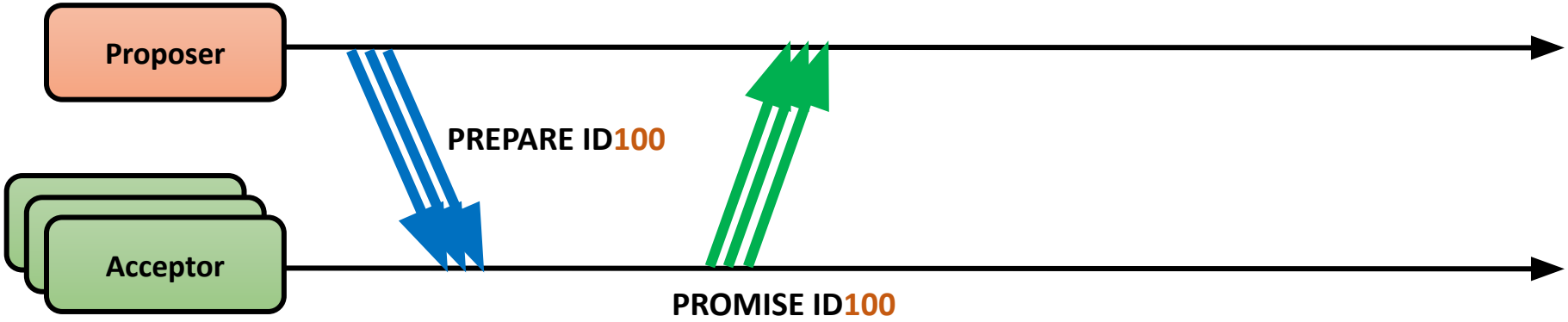
If majority of acceptors promise, no ID < IDp can pass the protocol

The acceptors agree on State 100

Acceptor received a PREPARE message with IDp:

- Did it promised to ignore requests with this IDp?
  - **YES:** Ignore
  - **NO:** Will promise to ignore any request lower than IDp
    - (?) Reply with PROMISE IDp

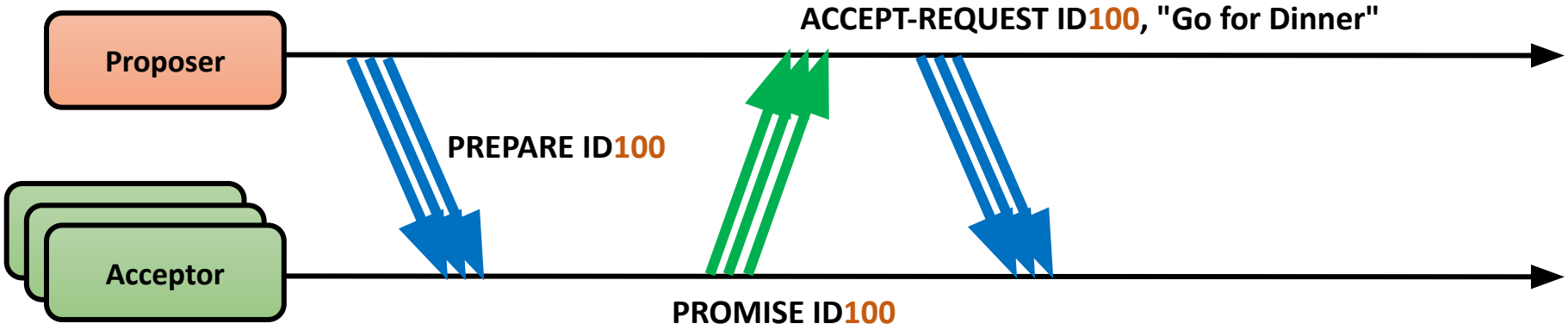
Paxos Algorithm



Proposer gets majority of PROMISE messages for a specific IDp:

- Sends **ACCEPT-REQUEST IDp**, **VALUE** to a majority (or all) of **Acceptors**
  - (?) It picks any value of its choice

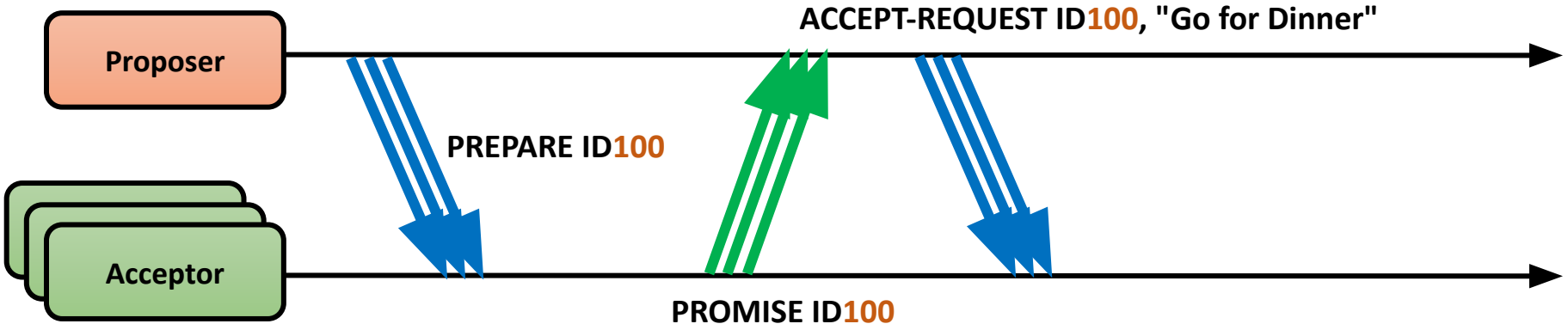
Paxos Algorithm



Proposer gets majority of PROMISE messages for a specific IDp:

- Sends **ACCEPT-REQUEST IDp**, **VALUE** to a majority (or all) of **Acceptors**
  - (?) It picks any value of its choice

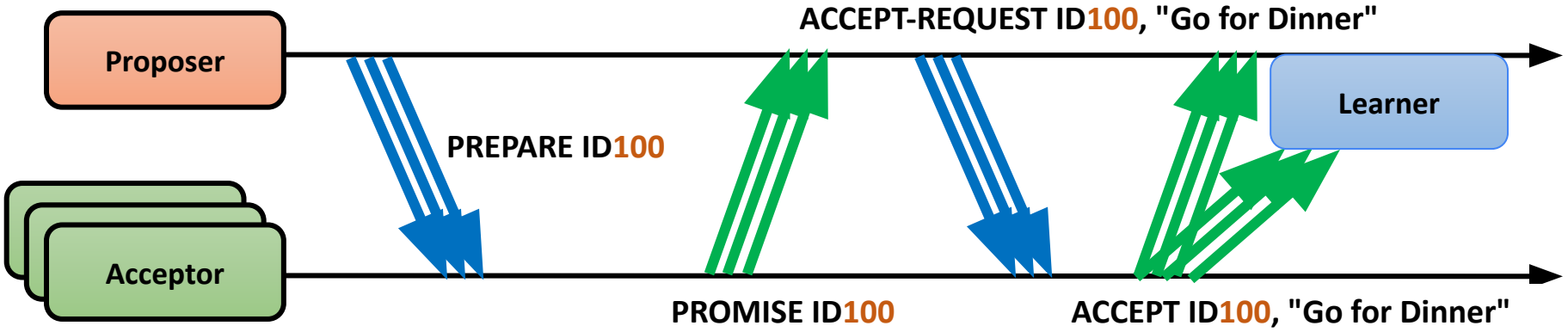
Paxos Algorithm



Acceptor receives an ACCEPT-REQUEST IDp, VALUE :

- Did it promised to ignore request with this IDp?
  - **YES:** Ignore
  - **NO:** Reply with **ACCEPT IDp, <u>VALUE</u>**; Also send it to all learners

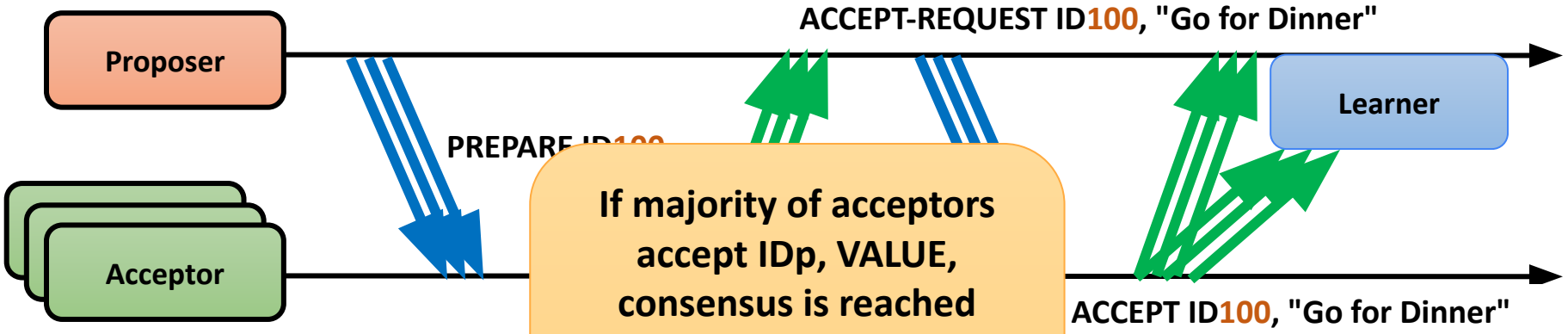
Paxos Algorithm



Acceptor receives an ACCEPT-REQUEST IDp, VALUE :

- Did it promised to ignore request with this IDp?
  - **YES:** Ignore
  - **NO:** Reply with **ACCEPT IDp, <u>VALUE</u>**; Also send it to all learners

Paxos Algorithm



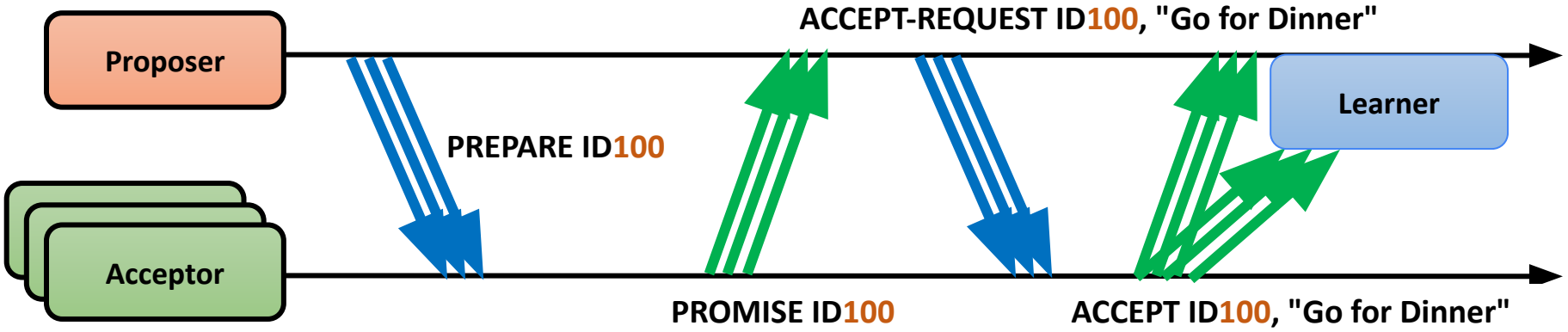
If majority of acceptors accept IDp, VALUE, consensus is reached

Consensus is reached on the value, not on IDp

Acceptor receives an ACCE

- Did it promised to ig
  - **YES:** Ignore
  - **NO:** Reply with **ACCEPT IDp, <u>VALUE</u>**; Also send it to all learners

Paxos Algorithm



Proposer or **Learner** gets ACCEPT message with IDp, VALUE:

- If a proposer/learner gets majority of accept for a specific IDp, they know that consensus is reached for the value (not IDp).

Practical Byzantine Fault Tolerance (PBFT)

Used to provide consensus in the presence of Byzantine faults.

Considers an asynchronous environment.

Utilize digital signature to to validate the identity of the senders.

Low overhead.

Comprises three sub-protocols called normal operation, view change, and checkpointing.

- Normal operation is executed when everything is running normally and no errors are in the system.
- View change is runs when a faulty leader node is detected in the system.
- Checkpointing is used to discard the old data from the system.

Comprises three phases - pre-prepare, prepare, and commit.

The protocol runs in rounds.

In each round, an elected leader node, called the primary node, handles the communication with the client.

The protocol progresses through all the three phases.

Participants in the PBFT protocol are called replicas

One of the replicas becomes primary as a leader in each round, and the rest of the nodes acts as backups.

in order to tolerate Byzantine faults, the minimum number of nodes required is $N = 3F + 1$, where N is the number of nodes and F is the number of faulty nodes

PBFT ensures Byzantine fault tolerance as long as the number of nodes in a system stays $N >= 3F + 1$

Eg: If your blockchain network has 4 Byzantine nodes, what is the minimum number of nodes that are required to ensure Byzantine fault tolerance using PBFT protocol?

> in order to tolerate Byzantine faults, the minimum number of nodes required is N = 3F + 1, where N is the number of nodes and F is the number of faulty nodes

Here, F = 4

Number of nodes required for Byzantine fault tolerance, N = 3F+1

N = 3*4+1 = 13.

So, if the network has at least 13 nodes, then it can tolerate Byzantine faults (misbehavior) of 4 nodes.

Advantages

- Provides immediate and deterministic transaction finality.
- Energy efficient as compared to PoW

Disadvantages

- Not very scalable - more suitable for consortium networks, instead of public blockchains.
- Faster than PoW protocols.
- Sybil attacks can be carried out

Proof of Stake (PoS)

Energy-efficient alternative to the PoW.

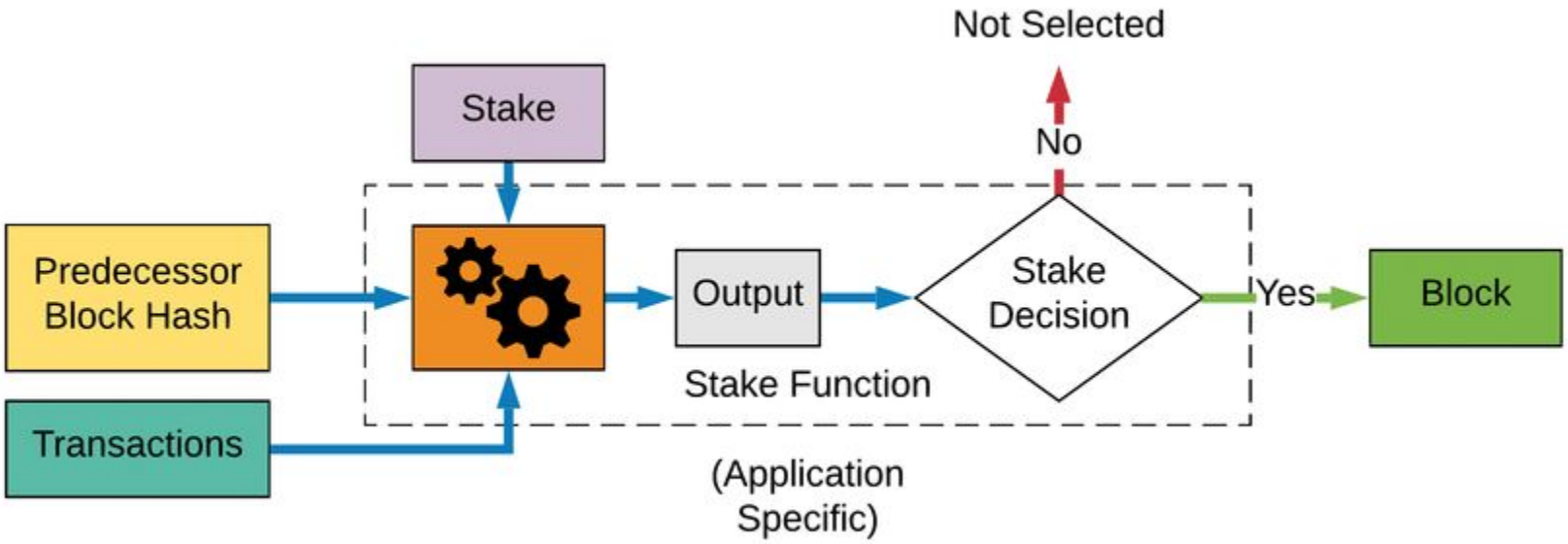
It was first proposed in Peercoin in August 2012.

'Stake' represents the number of coins (money) in the consensus protocol staked by a blockchain participant.

The chance of proposing the next block is directly proportional to the value staked by the participant.

Key idea is that if someone has a stake in the system, then they will not try to sabotage the system.

In PoS, miner is called either a validator, minter, or stakeholder

Proposers are rewarded either with transaction fees.

A hacker would need to own 51% of all the coins in the network, which is practically impossible.